

[Bitwise operations for beginners](#)

By [Errichto](#), 5 years ago, 

You can watch my Youtube video ([link](#)) with the same content as this blog. Anyway, enjoy.

Introduction

Let's learn bitwise operations that are useful in Competitive Programming. Prerequisite is knowing the binary system. For example, the following must be clear for you already.

$$13 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 1101(2) = 00001101(2) \quad 13 = 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 1101(2) = 00001101(2)$$

Keep in mind that we can pad a number with leading zeros to get the length equal to the size of our type size. For example, char has 8 bits and int has 32 bits.

Bitwise AND, OR, XOR

You likely already know basic logical operations like AND and OR. Using `if(condition1 && condition2)` checks if both conditions are true, while `OR (c1 || c2)` requires at least one condition to be true.

Same can be done bit-per-bit with whole numbers, and it's called bitwise operations. You must know bitwise AND, OR and XOR, typed respectively as `&`, `|`, `^`, each with just a single character. XOR of two bits is 1 when exactly one of those two bits is 1 (so, XOR corresponds to `!=` operator on bits). There's also NOT but you won't use it often. Everything is explained in [Wikipedia](#) but here's an example for bitwise AND. It shows that `53 & 28` is equal to 20.

$$53 = 110101$$

$$28 = 11100$$

$$110101$$

`& 11100` // imagine padding a shorter number with leading zeros to get the same length

$$010100 = 20$$

C++ code for experimenting

Shifts

There are also bitwise shifts `<<` and `>>`, not having anything to do with operators used with `cin` and `cout`.

As the arrows suggest, the left shift `<<` shifts bits to the left, increasing the value of the number. Here's what happens with `13 << 2` — a number 13 shifted by 2 to the left.

LEFT SHIFT

13 = 1101

(13 << 2) = 110100

RIGHT SHIFT

13 = 1101

(13 >> 2) = 11

If there is no overflow, an expression $x \ll b$ is equal to $x \cdot 2^b$, like here we had $(13 \ll 2) = 52$.

Similarly, the right shift $\gg$ shifts bits to the right and some bits might disappear this way, like bits 01 in the example above. An expression $x \gg b$ is equal to the floor of $x/2^b$. It's more complicated for negative numbers but we won't discuss it.

So what can we do?

2^k is just $1 \ll k$ or $1LL \ll k$ if you need long longs. Such a number has binary representation like 10000 and its AND with any number xx can have at most one bit on (one bit equal to 1). This way we can check if some bit is on in number xx . The following code finds ones in the binary representation of xx , assuming that $x \in [0, 10^9]$:

```
for(int i = 0; i < 30; i++) if((x & (1 << i)) != 0) cout << i << " ";
```

(we don't have to check $i=30$ because $2^{30} > x$)

And let's do that slightly better, stopping for too big bits, and using the fact that `if(value)` checks if value is non-zero in C++.

```
for(int i = 0; (1 << i) <= x; i++) if(x & (1 << i)) cout << i << " ";
```

Consider this problem: You are given $N \leq 20$ numbers, each up to 10^9 . Is there a subset with sum equal to given goal SS ?

It can be solved with recursion but there's a very elegant iterative approach that iterates over every number xx from 0 to $2^n - 1$ and considers xx to be a binary number of length n , where bit 1 means taking a number and bit 0 is not taking. Understanding this is crucial to solve any harder problems with bitwise operations. Analyze the following code and then try to write it yourself from scratch without looking at mine.

solution code

Two easy problems where you can practice iterating over all 2^N possibilities:

- <https://codeforces.com/problemset/problem/1097/B>

- <https://codeforces.com/problemset/problem/550/B>

Speed

Time complexity of every bitwise operation is $O(1)$. These operations are very very fast (well, popcount is just fast) and doing 10^9 of them might fit in 1 second. You will later learn about bitsets which often produce complexity like $O(n^2)$, good enough to pass constraints $n \leq 10^5$.

I will welcome any feedback. Coming next: popcount, bitsets, dp with bitmasks. I will also make a YT video on this. YT video link is at the top.

Part 2 [link](#)

[Bitwise operations 2 — popcount & bitsets](#)

By [Errichto](#), 5 years ago, 

Part 1 ([link](#)) introduces basic bitwise operations. This is part 2 and it's mainly about (in)famous bitsets and example problems. Also, see links to very useful advanced stuff at the bottom. EDIT: here's [video version](#) of this blog (on my Youtube channel).

Built-in functions

In C++, `__builtin_popcount(x)` returns popcount of a number — the number of ones in the binary representation of `xx`. Use `__builtin_popcountll(x)` for long longs.

There are also `__builtin_clz` and `__builtin_ctz` (and their long long versions) for counting the number of leading or trailing zeros in a **positive** number. Read more [here](#). Now, try to solve these two simple tasks in $O(1)$, then open the spoiler to check the solution:

Compute the biggest power of 2 that is a divisor of `x`. Example: $f(12) = 4$

Compute the smallest power of 2 that is not smaller than `x`. Example: $f(12) = 16$

While popcount is often needed, I rarely use the other two functions. During a contest, I would solve the two tasks above in $O(\log x)$ with simple while-loops, because it's easier and more intuitive for me. Just be aware that these can be done in $O(1)$, and use `clz` or `ctz` if you need to speed up your solution.

Motivation behind bitsets

Consider this problem: There are $N \leq 5000$ workers. Each worker is available during some days of this month (which has 30 days). For each worker, you are given a set of numbers, each from interval $[1, 30]$, representing his/her availability. You need to assign an important project to two workers but they will be able to work on the project only when they are both available. Find two workers that are best for the job — maximize the number of days when both these workers are available.

You can compute the intersection of two workers (two sets) in $O(30)$ by using e.g. two pointers for two sorted sequences. Doing that for every pair of workers is $O(N^2 \cdot 30)$, slightly too slow.

We can think about the availability of a worker as a binary string of length 30, which can be stored in a single int. With this representation, we can count the intersection size in $O(1)$ by using `__builtin_popcount(x[i] & x[j])`. The complexity becomes $O(N^2)$, fast enough.

What if we are given the availability for the whole year or in general for DD days? We can handle $D \leq 64$ in a single unsigned long long, what about bigger DD ?

We can split DD days into convenient parts of size 64 and store the availability of a single worker in an array of $D/64$ unsigned long longs. Then, the intersection can be computed in $O(D/64)O(D/64)$ and the whole complexity is $O(N^2 \cdot D/64)O(N^2 \cdot D/64)$.

code

So, we can simulate a long binary number with multiple unsigned long longs. The implementation isn't that bad but doing binary shifts would be quite ugly. Turns out all of this can be done with bitsets easily.

Bitsets

`bitset<365>` is a binary number with 365 bits available, and it supports most of binary operations. The code above changes into simple:

code

Some functions differ, e.g. `x.count()` instead of `__builtin_popcount(x)` but it's only more convenient. You can read and print binary numbers, construct a `bitset` from `int` or `string` `bitset<100> a(17);` `bitset<100> b("1010");`. You can even access particular bits with `b[i]`. Read more in C++ reference <https://en.cppreference.com/w/cpp/utility/bitset>.

Note that the size of the `bitset` must be a constant number. You can't read `nn` and then declare `bitset<n> john;`. If `nn` is up to 100100, just create `bitset<100>`.

The complexity of bitwise operations is $O(\text{size}/32)O(\text{size}/32)$ or $O(\text{size}/64)O(\text{size}/64)$, it depends on the architecture of your computer.

Problems

P1. Different numbers — You are given a sequence of $N \leq 10^7$ numbers, each from interval $[0, 10^9]$. How many different values appear in the sequence? Don't use `set` or `unordered_set` because they are quite slow.

solution

P2. Knapsack — You are given $N \leq 1000$ items, each with some weight w_i . Is there a subset of items with total weight exactly $W \leq 10^6$?

solution

P3. Triangles in a graph — Given a graph with $n \leq 2000$ vertices and $m \leq n \cdot (n-1)/2$ edges, count triples of vertices a, b, c such that there are edges $a-b$, $a-c$ and $b-c$.

hint

P4. Chef and Queries — <https://www.codechef.com/problems/CHEFQUE> (easy)

P5. Odd Topic — <https://www.codechef.com/AABH2020/problems/ODTPIC> (medium), thanks to **Not-Afraid** for the suggestion

P6. Funny Gnomes — <https://www.codechef.com/problems/SHAIKHGN> (hard)

Bonuses

1) $m \& (m-1)$ turns off the lowest bit that was set to 1 in a positive number m . For example, we get 2424 for $m=26$, as 1101011010 changes into 1100011000. [Explanation on quora](#)

2) A quite similar trick allows us to iterate efficiently over all submasks of a mask, [article on cp-algorithms / e-maxx](#). This article also explains why masks-submasks iteration is $O(3^n)$.

3) DP on broken profile (grid dp) — https://cp-algorithms.com/dynamic_programming/profile-dynamics.html

4) SOS dp (sum over subset)

— <https://codeforces.com/blog/entry/45223> & <https://www.youtube.com/watch?v=Lpvds8WpbWc&t=5m4s>

5) `_Find_next` function and complexity notation for bitsets — <https://codeforces.com/blog/entry/43718>

I will add links to some problems in online judges, feel free to suggest some in the comments. I think that bonuses 3 and 4 lack some explanation with drawings, maybe I will make some soon.

💡 [bitwise](#), [bits](#), [bitset](#)